

Savitribai Phule Pune University Second Year of Computer Engineering and Computer Science and Engineering (2024 Course)

PCC-201-COM: Data Structures

By Payal Hake

PCC-201-COM: Data Structures

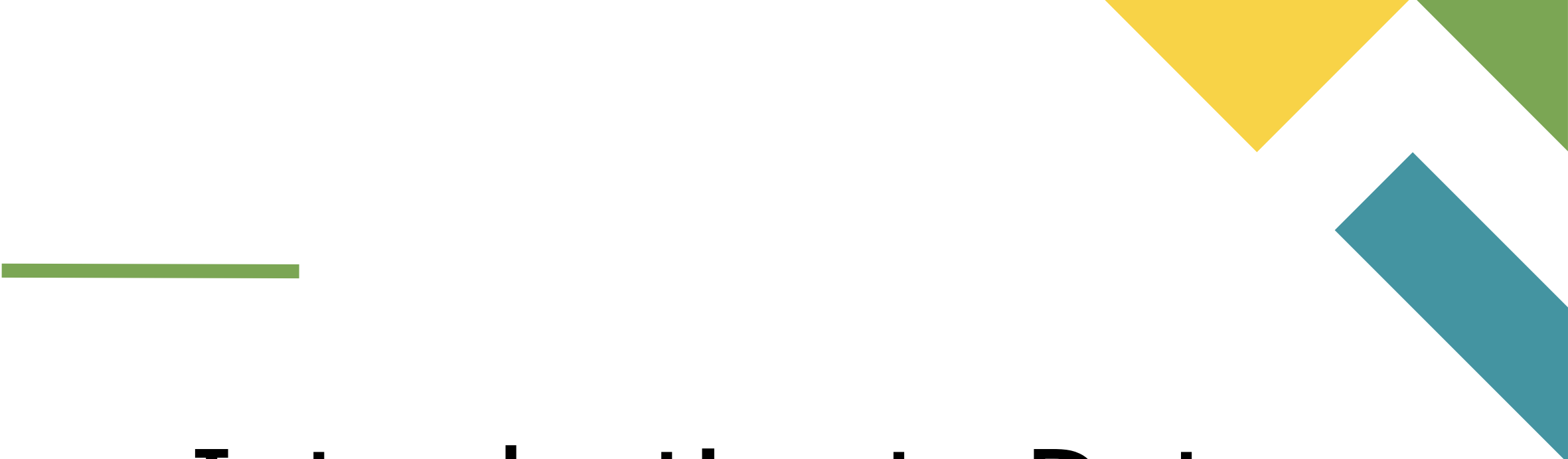
Teaching /scheme	Credits	Examination Scheme
Theory : 03 Hours/Week	03	CCE : 30 Marks End-Semester: 70 Marks

Unit I - Introduction to Data Structures and Algorithms (09 Hours)

- **Introduction:** Introduction to Data Structures: Abstract Data Types (ADT), Linear and Non-linear, Static and Dynamic, Persistent and Ephemeral data structures.
- **Algorithms:** Space complexity, Time complexity, Asymptotic notation- Big-O, Theta and Omega, finding complexity using step count method, Analysis of programming constructs-Linear, Quadratic, Cubic, Logarithmic.
- **Algorithmic Strategies:** Introduction to algorithm design strategies- Divide and Conquer, and Greedy strategy.
- **Case Study:** E-commerce Product Sorting using Divide and Conquer strategy
Google Calendar application using Greedy strategy.

Time required to cover UNIT 1 topics

Topics	# Hours
Introduction: Introduction to Data Structures: Abstract Data Types (ADT), Linear and Non-linear, Static and Dynamic, Persistent and Ephemeral data structures.	1
Algorithms: Space complexity, Time complexity, Asymptotic notation- Big-O, Theta and Omega, finding complexity using step count method.	1
Analysis of programming constructs- Linear, Quadratic, Cubic, Logarithmic.	1
Algorithmic Strategies: Introduction to algorithm design strategies- Divide and Conquer, and Greedy strategy.	1
Case Study: E-commerce Product Sorting using Divide and Conquer strategy, Google Calendar application using Greedy strategy.	1
Remaining hours can be utilized for other units.	



Introduction to Data Structures

Data Structures



- Data Structures are the main part of many computer science algorithms.
- They enable the programmers to handle the data in an efficient way.
- It plays an important role in enhancing the performance of a software or a program as the main function of the software is to store and retrieve the user's data as fast as possible.
- Choosing the appropriate data structure for a program is the most difficult task for a programmer.

Abstract Data Types (ADT)



- A conceptual model that defines a set of operations and behaviors for a data structure, **without specifying how these operations are implemented** or how data is organized in memory.
- The definition of ADT only mentions what **operations are to be performed** but **not how** these operations will be implemented.
- The process of providing only the essentials and hiding the details is known as abstraction.

Stack ADT (contd...)

- The Stack is a linear data structure that follows the LIFO (Last In, First Out) principle.
- It allows elements to be added and removed only from one end, called the top of the stack.
- It should also support the following operations:
 - push():** Insert an element at one end of the stack called the top.
 - pop():** Remove and return the element at the top of the stack, if it is not empty.
 - peek():** Return the element at the top of the stack without removing it, if the stack is not empty.

Stack ADT

peek(): Return the element at the top of the stack without removing it, if the stack is not empty.

size(): Return the number of elements in the stack.

isEmpty(): Return true if the stack is empty; otherwise, return false.

isFull(): Return true if the stack is full; otherwise, return false. Only relevant for fixed-capacity stacks (e.g., array-based).

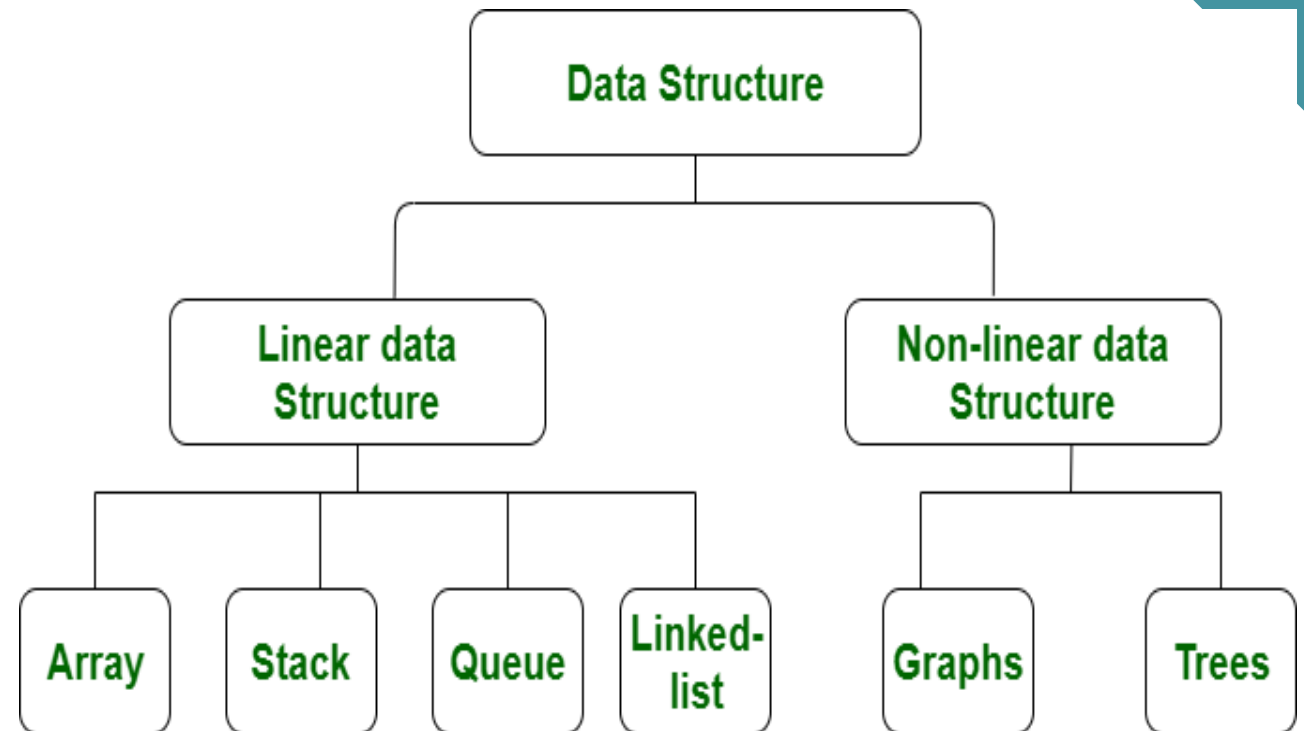
Linear and Non-linear data structures

- **Linear data structures:**

- Arrange elements sequentially, like a list.

- **Non-linear structures:**

- Allow for more complex relationships, such as hierarchical or interconnected arrangements.



Static and Dynamic data structures

➤ Static data structure

- The size of the structure is fixed.
- The content of the data structure can be modified but without changing the memory space allocated to it.
- Example: Array

40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

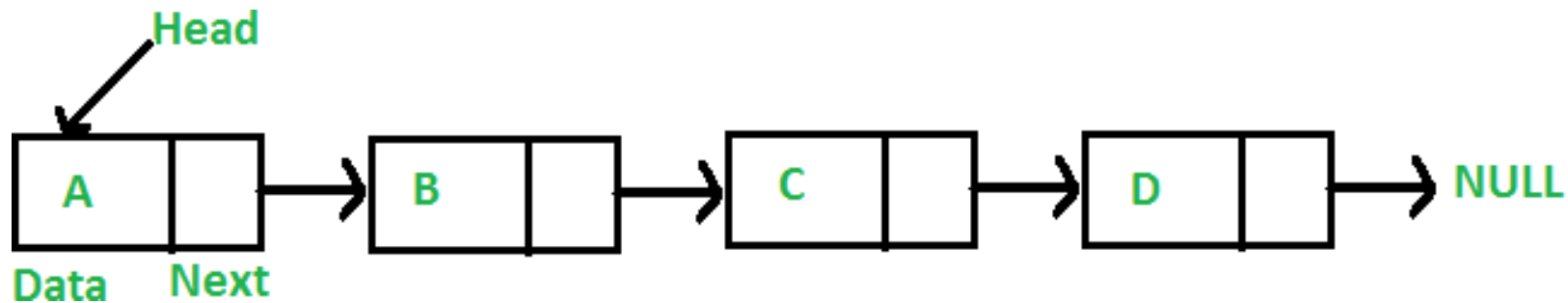
First Index = 0

Last Index = 8

Static and Dynamic data structures

➤ Dynamic data structure

- The size of the structure is not fixed.
- It can be modified during the operations performed on it.
- Dynamic data structures are designed to facilitate change of data structures in the run time.
- Example: Linked List



Persistent and Ephemeral data structures

➤ Ephemeral data structures

- Ephemeral – means short-lived, temporary
- All the data structures discussed here so far are non-persistent (or ephemeral).

➤ Persistent data structures

- Persistent – means permanent
- Always preserves the previous version of itself when it is modified.
- They can be considered as 'immutable' as updates are not in-place.
- A data structure is **partially persistent** if all versions can be accessed but only the newest version can be modified.
- **Fully persistent** if every version can be both accessed and modified.
- **Confluently persistent** is when we merge two or more versions to get a new version.

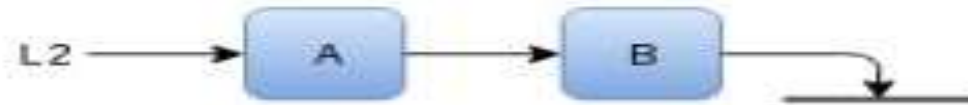
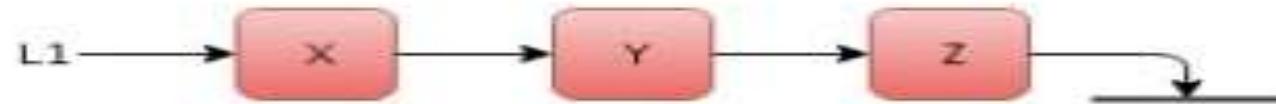
Persistent data structures: Example (contd...)

Linked List Concatenation

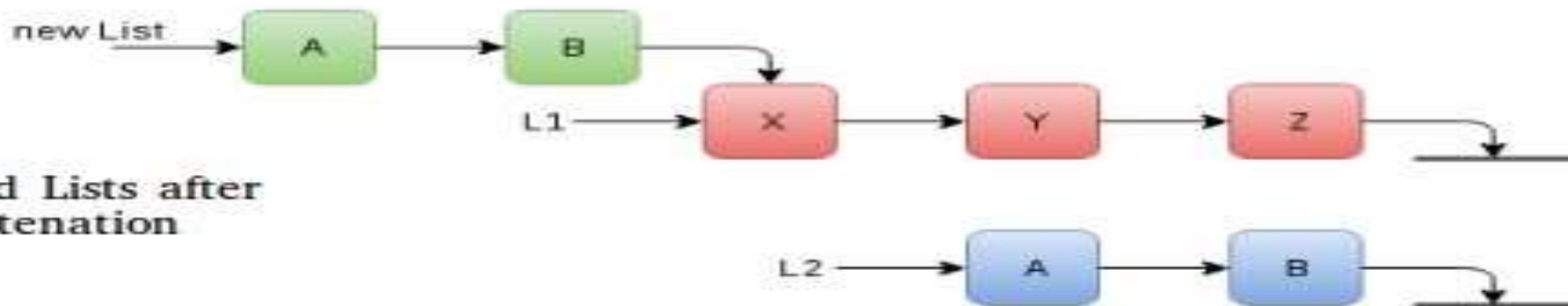
- Consider the problem of concatenating two singly linked lists.
- We need to keep the versions, i.e., we should be able to access original list.
- 2 methods
 - i. Make a copy of every node and do the connections.
 - ii. More efficient way in time and space, involves traversal of only one of the two lists and fewer new connections.

Persistent data structures: Example

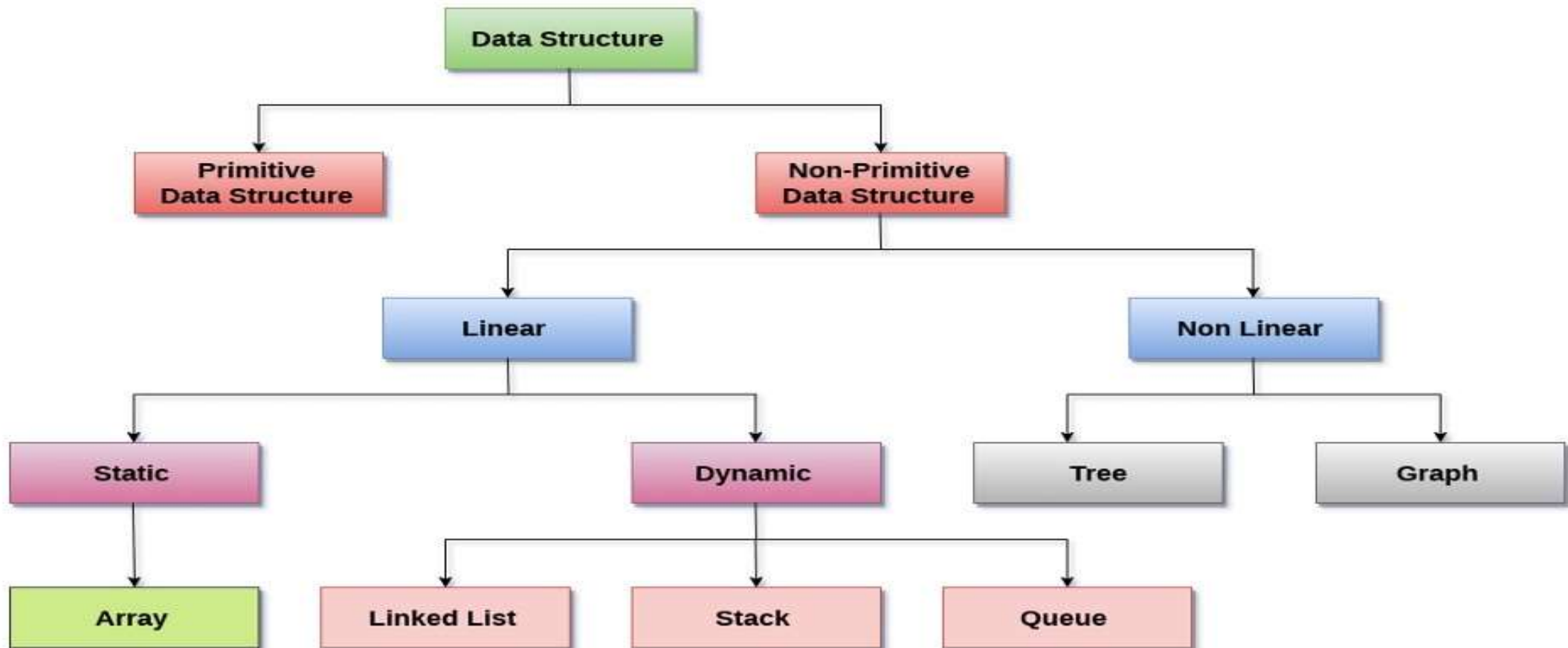
Given linked lists :
Concatenate L2 into L1

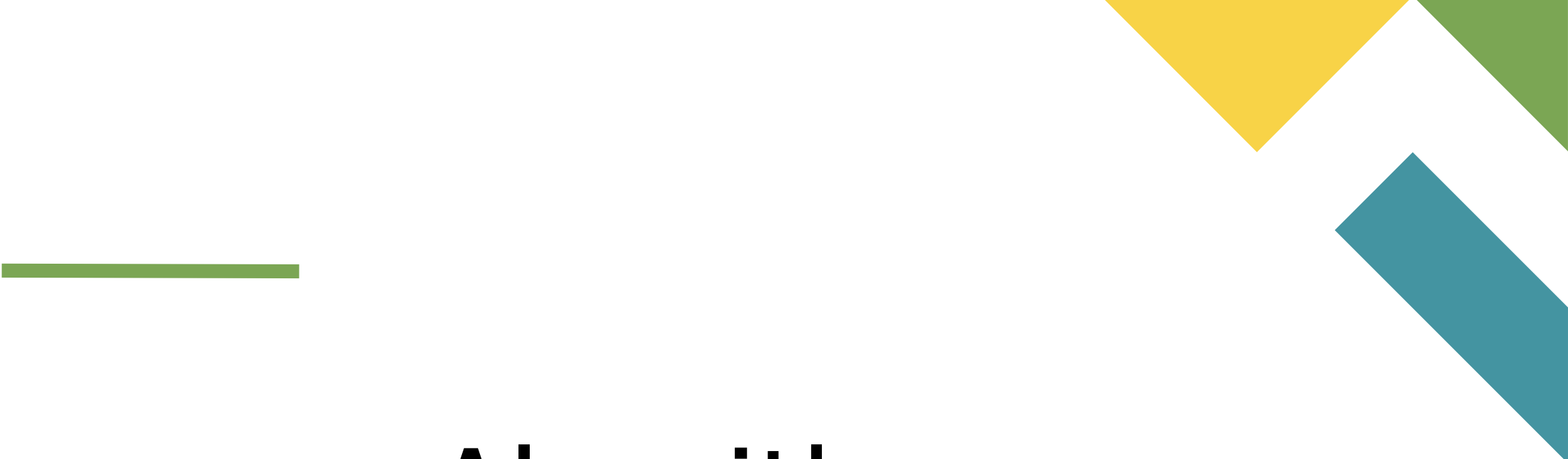


Linked Lists after
concatenation



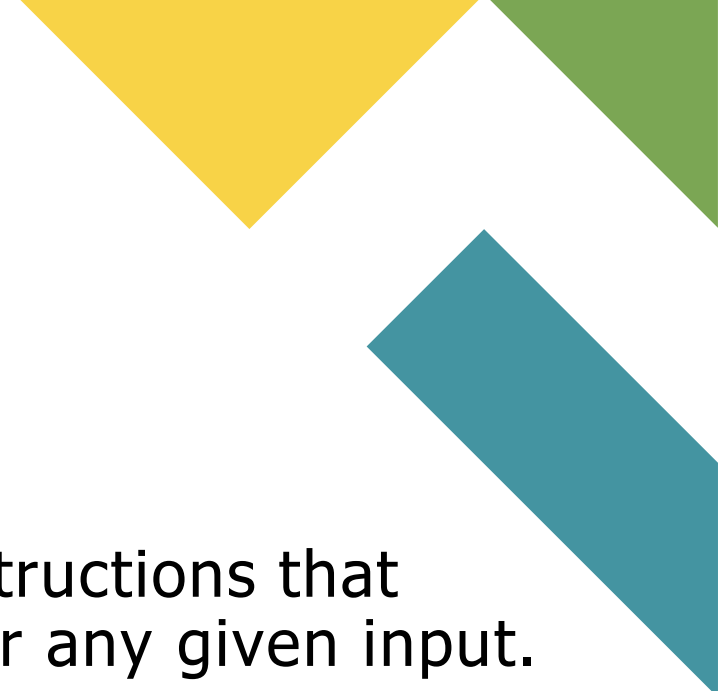
Data Structure Classification





Algorithms

Algorithms



- An algorithm
 - An algorithm is a general step-by-step set of instructions that solves a problem and gives the correct output for any given input.
- A program
 - A program is an algorithm written in a programming language.
 - The same algorithm can be turned into different programs depending on the language and the programmer.

What Is Algorithm Analysis?

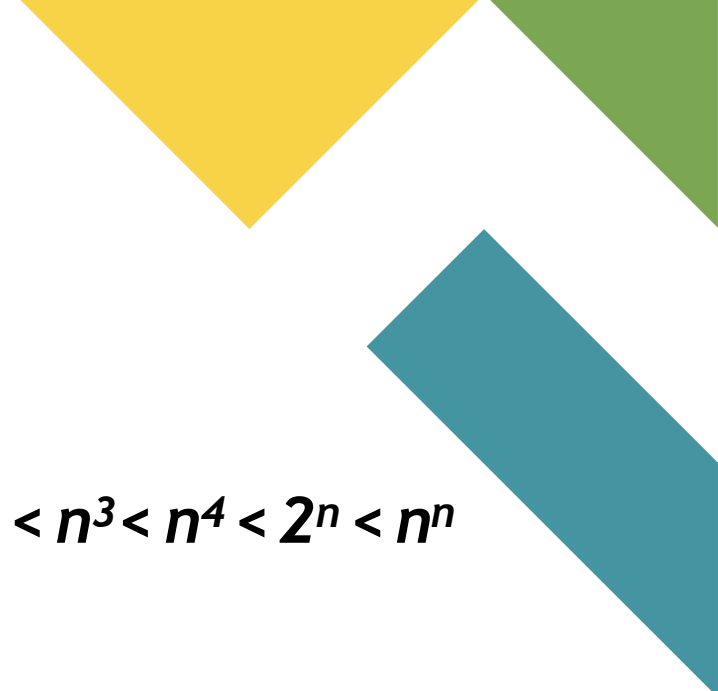


- It is very common for beginning computer science students **to compare their programs with one another.**
- When two programs solve the same problem but look different, is one program better than the other?
- Algorithm analysis is concerned with **comparing algorithms** based upon the **amount of computing resources that each algorithm uses.**
 - **Space complexity**
 - **Time complexity**

What is Performance Analysis of an algorithm?

- Suppose if we want to go from city "A" to city "B", there can be many ways. We can go by flight, bus, train and also bicycle. Depending on the **availability** and **convenience**, we choose the one which is suitable.
- Similarly, in computer science, when we have more than one algorithm to solve a problem, we need to **select the best one. Performance analysis helps to select the best algorithm.**
- The idea is to measure **order of growths** in terms of **input size**.

Order of growths



$c < \text{LogLog}n < \text{Log}n < n^{1/3} < n^{1/2} < n < n \text{Log}n < n^2 < n^2 \text{Log}n < n^3 < n^4 < 2^n < n^n$

Here c is a constant

Space complexity



- Space complexity is not the actual memory used by the program, but the amount of memory an algorithm needs based on the input size (n).
- It measures the extra (auxiliary) space required to solve the problem.

Consider the following example

```
int square(int a)
{
    return a * a;
}
```

It requires 2 bytes of memory to store variable 'a' and another 2 bytes of memory is used for **return value**.

Totally it requires 4 bytes of memory to complete its execution. And this 4 bytes of memory is fixed for any input value of 'a'. This space complexity is said to be **Constant Space Complexity**.

$$\begin{aligned} SC &= O(K) \\ &= O(1) \end{aligned}$$

Consider the following example

```
int sum(int A[ ], int n)
{ int sum = 0, i;
  for(i = 0; i < n; i++)
    sum = sum + A[i];
  return sum; }
```

'**n*2**' bytes of memory to store array variable '**a[]**'
2 bytes of memory for integer parameter '**n**'
4 bytes of memory for local integer
variables '**sum**' and '**i**' (2 bytes each)
2 bytes of memory for **return value**.

totally it requires ' $2n+8$ ' bytes of memory to complete its execution. Here, the total amount of memory required depends on the value of ' n '. As ' n ' value increases the space required also increases proportionately. This type of space complexity is said to be *Linear Space Complexity*.

Time complexity



- Time complexity is not the exact time taken, but how the time to run an algorithm increases with input size (n).
- It tells us the **execution or running time** of the algorithm.

Asymptotic Notations



**Asymptotic Notation:
Big-O, Theta, Big-Omega**

Big-O Notation (O) (contd...)

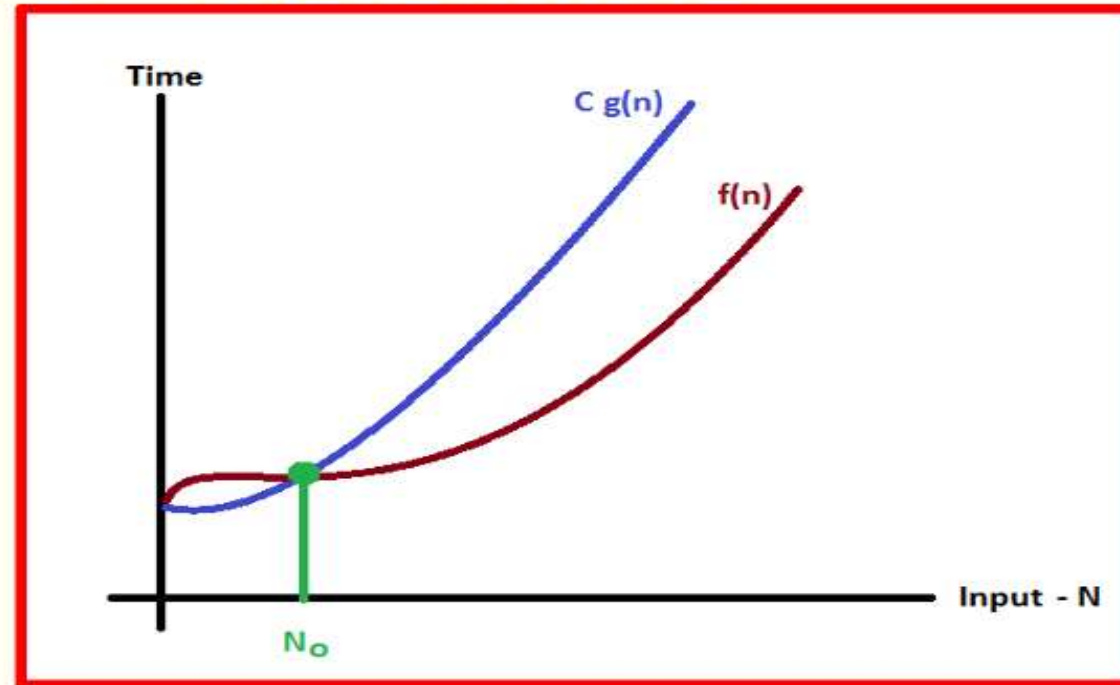
- Used to define the **upper bound** of an algorithm in terms of time Complexity.
- Indicates the **maximum time** required by an algorithm for all input values.
- Describes the worst case of an algorithm time complexity.
- Definition: **$f(n) = O(g(n))$**

Consider function $f(n)$ as time complexity of an algorithm and $g(n)$ is the most significant term.

If $f(n) \leq C g(n)$ for all $n \geq n_0$, $C > 0$ and $n_0 \geq 1$. Then we can represent $f(n)$ as $O(g(n))$.

Big-O Notation (O)

Consider the following graph drawn for the values of $f(n)$ and $C g(n)$ for input (n) value on X-Axis and time required is on Y-Axis



In above graph after a particular input value n_0 , always $C g(n)$ is greater than $f(n)$ which indicates the algorithm's upper bound.

Omega Notation (Ω) (contd...)

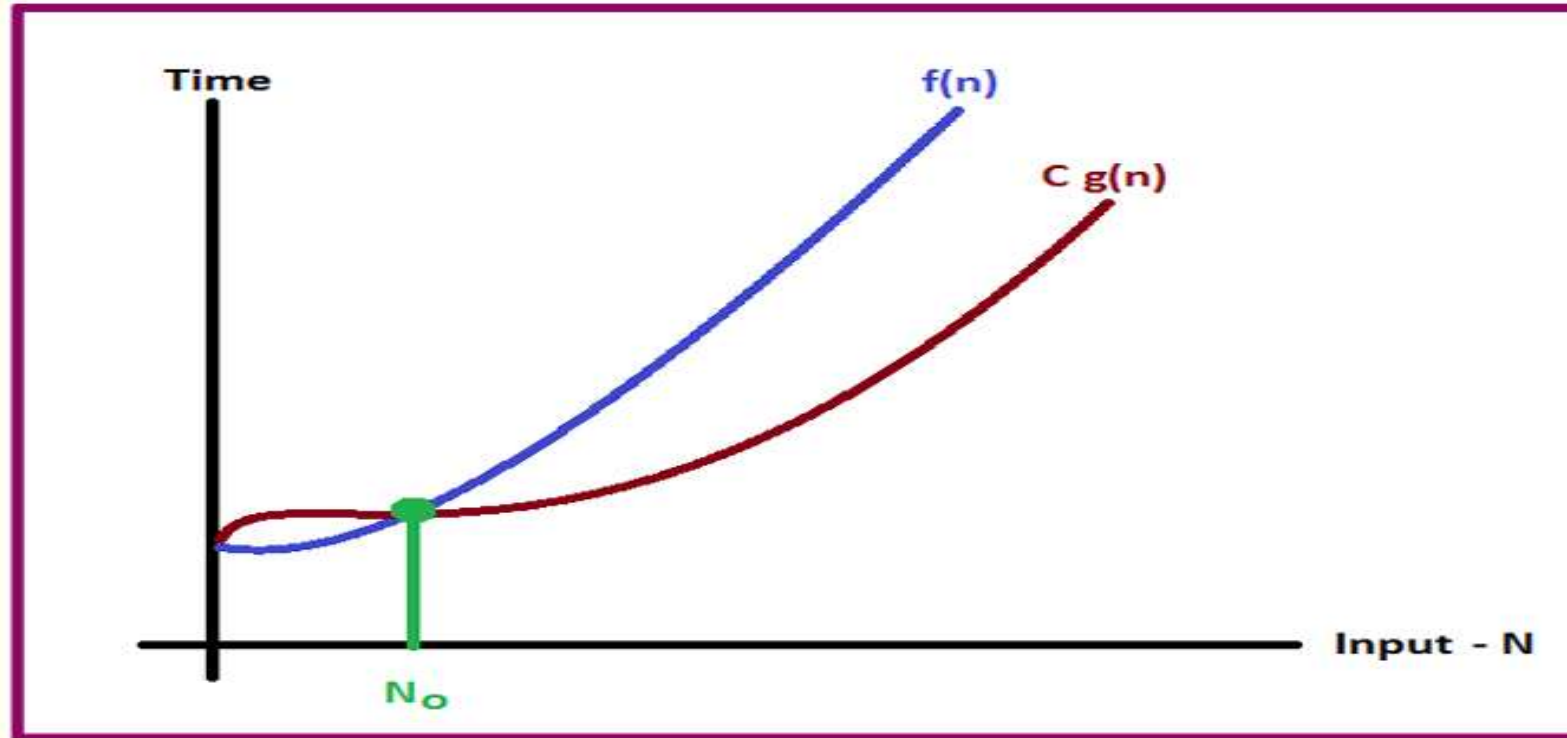
- Used to define the **lower bound** of an algorithm in terms of time Complexity.
- Always indicates the **minimum time** required by an algorithm for all input values.
- Describes the best case of an algorithm time complexity.
- Definition: **$f(n) = \Omega(g(n))$**

Consider function $f(n)$ as time complexity of an algorithm and $g(n)$ is the most significant term.

If $f(n) \geq C g(n)$ for all $n \geq n_0$, $C > 0$ and $n_0 \geq 1$.

Then we can represent $f(n)$ as $\Omega(g(n))$.

Omega Notation (Ω)



In above graph after a particular input value n_0 , always $C g(n)$ is less than $f(n)$ which indicates the algorithm's lower bound.

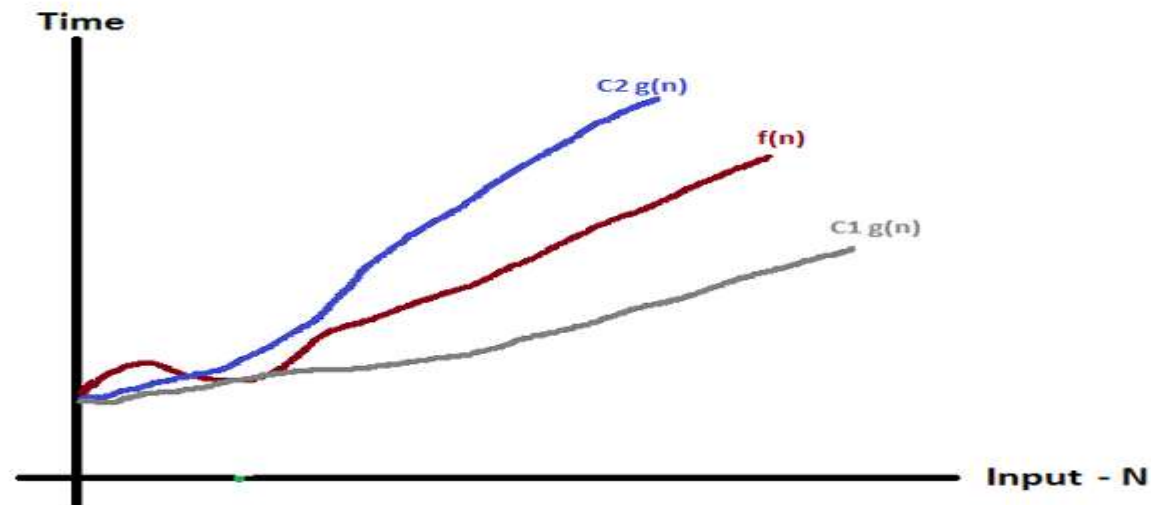
Theta Notation (Θ) (Contd...)

- Used to define the **average bound** of an algorithm in terms of time Complexity.
- Always indicates the **average time required** by an algorithm for all input values.
- Describes the average case of an algorithm time complexity.
- Definition: **$f(n) = \Theta(g(n))$**

Consider function $f(n)$ as time complexity of an algorithm and $g(n)$ is the most significant term.

If $C_1 g(n) \leq f(n) \leq C_2 g(n)$ for all $n \geq n_0$, $C_1 > 0$, $C_2 > 0$ and $n_0 \geq 1$.
Then we can represent $f(n)$ as $\Theta(g(n))$.

Theta Notation (Θ)



In above graph after a particular input value n_0 , always $C_1 g(n)$ is less than $f(n)$ and $C_2 g(n)$ is greater than $f(n)$ which indicates the algorithm's average bound.

Examples:



1] $f(n) = 4n^2 + 3n + 6$

2] $f(n) = 100 + 5n^2 + \sqrt{n}$

Examples:

$$1] f(n) = 4n^2 + 3n + 6$$

$$= n^2 + n + 1$$

$$= O(n^2)$$

$$2] f(n) = 100 + 5n^2 + \sqrt{n}$$

$$= 1 + n^2 + \sqrt{n}$$

$$= O(n^2)$$

Finding complexity using step count method

The Step Count Method is used to analyze the time complexity of an algorithm by counting the number of basic operations it performs.

Key Concepts:

- **Step:** A basic operation (e.g., assignment, arithmetic, comparison, function call).
- **Frequency:** How many times an operation is executed.
- **Time Complexity:** Total number of steps, usually expressed in Big-O notation (e.g., $O(n)$, $O(n^2)$).

Working Steps of step count method

1. Identify Basic Steps

- Find operations like assignments, loops, conditions, etc.

2. Assign Step Counts

- For example:
 - Assignment = 1 step
 - Loop = number of iterations $\times$ steps inside

3. Calculate Frequency

- Count how often each operation runs (especially inside loops).

4. Compute Total Steps

- Multiply step count $\times$ frequency
- Add all steps to get the total time complexity

5. Final Result:

- Express the total step count as a **function of input size** $\rightarrow$ then simplify using **Big-O notation**.

Analysis of Linear Search algorithm

Instruction	Step	Frequency	Total Steps
Algorithm Sum(a, n)	-	-	-
{	-	-	-
sum = 0;	1	1	1
for i = 1 to n do		n+1	n+1
{	-	-	-
sum = sum + a[i];	1	n	n
}	-	-	-
return sum;	1	1	1
}	-	-	-
Total steps =	-	-	2n+3

Finding complexity using step count method



- Therefore, the time complexity of this algorithm is **$O(n)$** .
- Reference: Horowitz Sahani :Algorithms

Examples:

```
1] for i in range(n):  
    print(i)
```

```
2] for i in range(n):  
    for j in range(n):  
        print(i, j)
```

```
3] a = 5  
   b = 10  
   print(a + b)
```

Examples:



**1] for i in range(n):
 print(i)**

Step Count: n steps (1 print per iteration)
Time Complexity: $O(n)$

**3] a = 5
 b = 10
 print(a + b)**

Step Count: Constant number of steps
Time Complexity: $O(1)$

**2] for i in range(n):
 for j in range(n):
 print(i, j)**

Step Count: $n * n = n^2$ steps
Time Complexity: $O(n^2)$

Analysis of programming constructs



-
- Linear
 - Quadratic
 - Cubic
 - Logarithmic

Constant Time: $O(1)$

- An algorithm runs in **constant time** if its execution time **does not depend on the size of the input (n)**.

$A=A+B$

- A single operation
- Executes in fixed time, regardless of input size

Why $O(1)$?

- No loops or recursion
- No input-dependent operations
- Always takes the **same time** to execute

Real-Life Analogy:

Flipping a light switch – takes the same time no matter the room size.

Linear Time: $O(n)$

- Time grows **linearly with input size**
- **Execution time is directly proportional to n**

for ($i=1 ; i \leq n ; i++$)

$a = a * b;$

- Runs n times
- Total time = $O(n)$

Why $O(n)$?

- Each iteration = $O(1)$
- Repeated n times $\rightarrow$ Total = $O(n)$

Real-Life Analogy:

- Reading n pages of a book $\rightarrow$ Takes time proportional to number of pages

Quadratic Time: $O(n^2)$

- Execution time grows **proportionally to the square of input size (n^2)**

```
for(i=1;i<=n;++i)
    for(j=1;j<=n;j++)
        a=a*b
```

- Outer loop = n times
- Inner loop = n times
- Total operations = $n \times n = O(n^2)$

Why $O(n^2)$?

- Nested loops, each dependent on n
- Time increases rapidly as n grows

Real-Life Analogy:

- Comparing all students with each other in a class of n students → Requires n^2 comparisons

Cubic Time: $O(n^3)$

- An algorithm has **cubic time complexity** when **execution time grows proportionally to the cube of input size (n^3)**.

```
for(i=1;i<=n;++i)
  for(j=1;j<=n;j++)
    for(k=1;k<=n;k++)
      a=a*b
```

Why $O(n^3)$?

- Three nested loops
- Each loop runs n times
- Total operations = $n \times n \times n = O(n^3)$

Real-Life Analogy:

- Checking every combination of 3 items from n choices → Time grows very fast as n increases

Logarithmic Time: $O(\log n)$

- An algorithm has logarithmic time complexity when **execution time grows slowly, in proportion to $\log n$.**

- Input is doubled each step
- Runs in **$O(\log n)$** time

Why $O(\log n)$?

- Each step reduces problem size exponentially
- Few steps needed even for large n

Real-Life Analogy:

- Binary Search in a sorted list → Cuts the search range in half each time

Example 1:

```
for(i=1; i<=n; i=i*2)  
    a=a*b
```

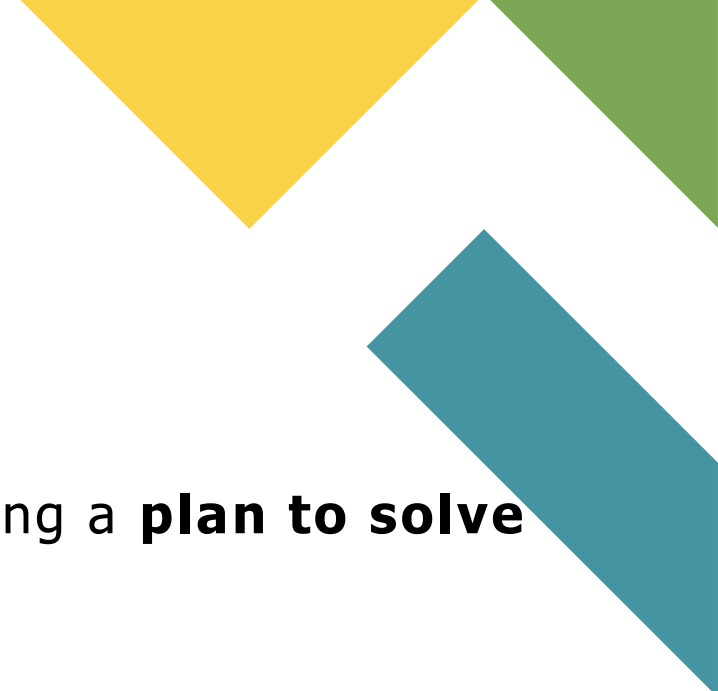
Example 2:

```
for(i=n; i>=1; i=i/2)  
    a=a*b
```



Algorithm Design Strategies

Algorithm design strategies



- In computer science, designing an algorithm is like creating a **plan to solve a problem efficiently**.
- Complex problems can be broken down and tackled using the right design technique.

Popular Algorithm Design Strategies:

- Divide and Conquer
- Greedy Strategy
- (Focus on Divide and Conquer here)

Divide and Conquer



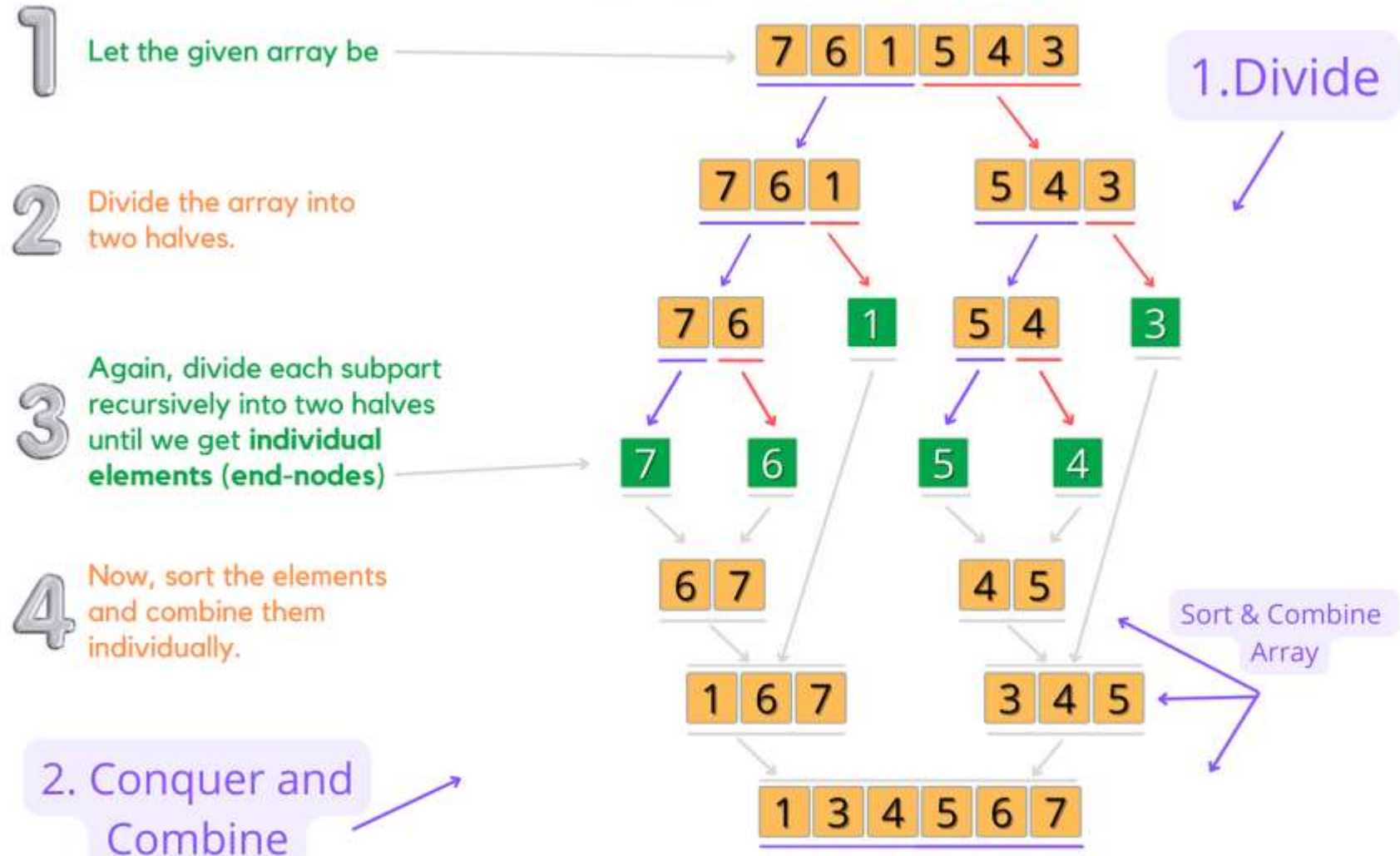
- Break the problem into **smaller sub-problems**, solve each one, and then combine the results to form a final solution.
- It's a **recursive approach** that works very well for problems with optimal substructure.

Steps in Divide and Conquer:

- Divide – Split the problem into smaller parts.
- Conquer – Solve each sub-problem (usually recursively).
- Combine – Merge the solutions of sub-problems into one.

Examples: Binary Search, Merge Sort, Quick Sort

Problem: Sort an Array using **Divide & Conquer** technique



Conquer the subproblems by solving them recursively. Combine the solution for subproblems into the solution for original problem.

Binary search



- Search for an element in a **sorted list**

Process:

- Compare the target with the middle element
- If smaller → Search left half
- If greater → Search right half
- Repeat until found or no elements left
- Each step discards half of the elements → Efficient

Binary search

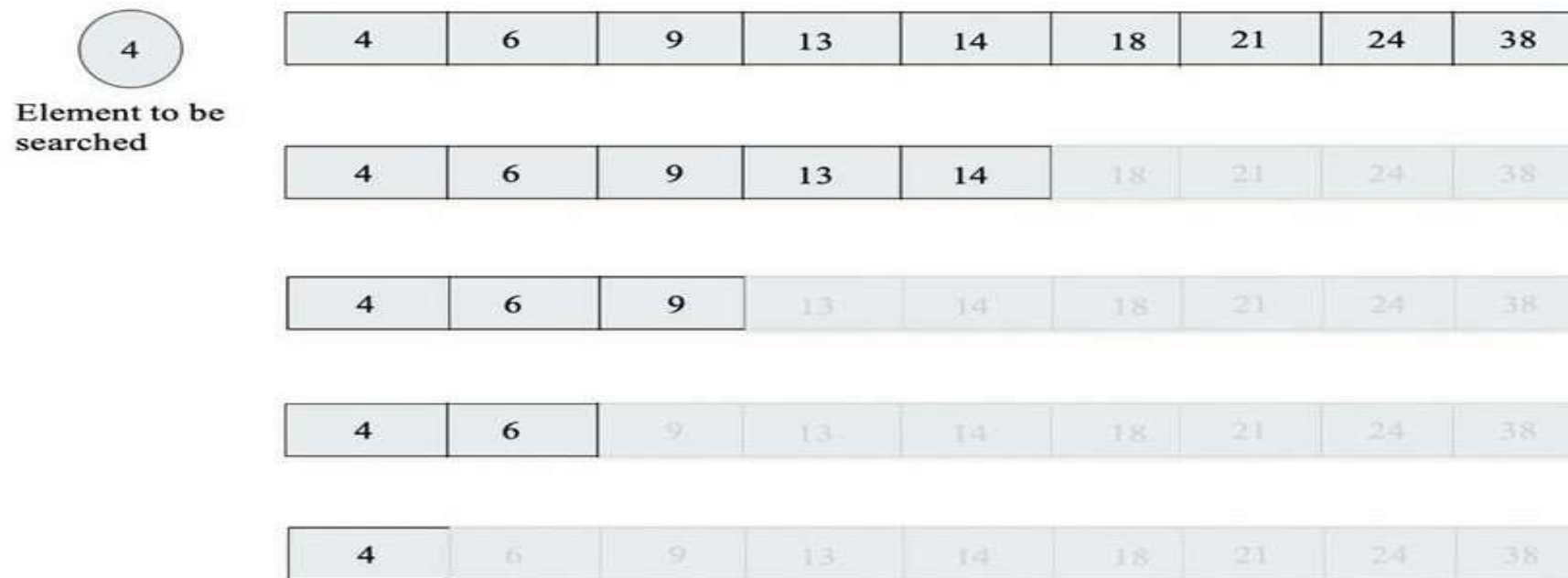


Figure 3.2: The process of searching for an element using a binary search algorithm

Figure taken from Reference book1.

Binary search

Initially

Find Key = 23 using Binary Search

	0	1	2	3	4	5	6	7	8	9
arr[] =	2	5	8	12	16	23	38	56	72	91



Binary search

Binary Search										
Search 23	0	1	2	3	4	5	6	7	8	9
	2	5	8	12	16	23	38	56	72	91
23 > 16 take 2 nd half	L=0	1	2	3	M=4	5	6	7	8	H=9
	2	5	8	12	16	23	38	56	72	91
23 < 56 take 1 st half	0	1	2	3	4	L=5	6	M=7	8	H=9
	2	5	8	12	16	23	38	56	72	91
Found 23, Return 5	0	1	2	3	4	L=5, M=5	H=6	7	8	9
	2	5	8	12	16	23	38	56	72	91



Greedy strategy

What is the Greedy Method?

- The **Greedy method** is an algorithmic approach used to solve **optimization problems**.
- It builds a solution **step-by-step**, choosing the **best possible option** at each stage, **without reconsidering** previous decisions.

What are Optimization Problems?

- Problems that require either a **maximum** or a **minimum** result.
- Examples:
 - **Maximum profit**
 - **Minimum cost**
- These problems often have multiple solutions.

Greedy strategy

Feasible and Optimal Solutions

- **Feasible Solution:** A solution that satisfies **all the given constraints** of the problem.
- **Optimal Solution:** A feasible solution that gives the **best possible result** (either minimum or maximum depending on the problem).

A problem can have:

- Multiple solutions
- Multiple feasible solutions
- **Only one optimal solution**

Greedy strategy

When to Use Greedy Algorithms?

Greedy algorithms are suitable when:

1. The problem has the **Greedy Choice Property**:
 - A global optimal solution can be arrived at by choosing local optimums.
2. The problem has **Optimal Substructure**:
 - An optimal solution of the problem contains optimal solutions to its subproblems.

GreedyAlgorithm(P):

1. Initialize solution set $S = \{\}$
2. While there are choices left in P:
 - a. Select the best feasible choice x from remaining options
 - b. If x is feasible, add x to S
3. Return S as the final solution

Examples of Greedy Algorithm Applications:

Problem	Application
Kruskal's Algorithm	Minimum Spanning Tree
Prim's Algorithm	Minimum Spanning Tree
Dijkstra's Algorithm	Shortest Path
Fractional Knapsack Problem	Maximize value
Traveling Salesperson Problem (TSP)	Approximate shortest path



Case Studies

Case Study



- E-commerce Product Sorting using Divide and Conquer strategy
- Google Calendar application using Greedy strategy

E-commerce Product Sorting using Divide and Conquer strategy (contd...)

- This method offers predictable performance and is well-suited for large datasets common in e-commerce.

- How it works:

1. Divide:

The initial list of products (unsorted) is recursively divided into smaller sub-lists until each sub-list contains only one element (which is inherently sorted).

E-commerce Product Sorting using Divide and Conquer strategy (contd...)

2. Conquer:

The sub-lists are then sorted using Merge Sort or Quick Sort.

3. Combine:

The sorted sub-lists are merged back together in a sorted manner, eventually resulting in a fully sorted list of all products.

Benefits for E-commerce



- **Efficiency:**

Divide and conquer algorithms like Merge Sort and Quick Sort have a time complexity of $O(n \log n)$, making them efficient for large product catalogs.

- **Scalability:**

This approach can handle a large number of products without significant performance degradation.

- **Predictable performance:**

The time complexity of these algorithms is consistent, making it easier to optimize performance.

Google Calendar application using Greedy strategy (contd...)

- A greedy strategy in Google Calendar could involve prioritizing the scheduling of events based on the earliest start times, aiming to maximize the number of events that can be fit into a given time frame.
- A greedy approach might be applied to scheduling events in Google Calendar as follows:

1.Event Prioritization:

SortbyStartTime:

Events could be sorted based on their earliest start times. This prioritizes fitting in events that begin sooner, potentially leaving more room for subsequent events.

Google Calendar application using Greedy strategy (contd...)

2. Scheduling Process:

Start with the Earliest Event: Begin by selecting the event with the earliest start time.

Check for Conflicts: Before adding an event to the calendar, check for any overlaps with existing events. If no conflict exists, the event can be scheduled.

Repeat for Remaining Events: Continue this process, moving to the next earliest event and checking for conflicts, until all events have been considered.

Google Calendar application using Greedy strategy (contd...)

Let's say you have the following events:

Event A: 9:00 AM - 10:00 AM

Event B: 9:30 AM - 10:30 AM

Event C: 10:30 AM - 11:30 AM

Event D: 11:00 AM - 12:00 PM

- A greedy approach might first schedule Event A, then Event C or D.
- OR Event B followed by D.

References

- Data structures and algorithms in python by Michael T. Goodrich, ISBN-13: 978-1118290279, ISBN-10: 1118290275, Publisher: Wiley; 1st edition (March 18, 2013).
- Problem Solving with Algorithms and Data Structures Using Python by Bradley N Miller and David L. Ranum. ISBN-13: 978-1590282571, ISBN-10: 1590282574, Publisher: Franklin, Beedle & Associates; 2nd edition (August 22, 2011).
- Hands-On DataStructures and Algorithms with Python: Write complex and powerful code using the latest features of Python 3.7, 2nd Edition by Dr. Basant Agarwal, Benjamin Baka. ISBN: 9781788991933, 2018.
- Core Python Programming-R. Nageswara Rao, ISBN-10: 9789351199427, ISBN-13: 978 9351199427, Willy; 1st edition (January 1, 2016).
- Horowitz, Sahani and Rajasekaran, Fundamentals of Computer Algorithms



THANK YOU...